

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий
Кафедра прикладной математики

Отчет защищен с оценкой _____

Преподаватель _____
(подпись)

«___» _____ 2026 г.

Отчет
По лабораторной работе №4

«Функции»

по дисциплине «Программирование – 2 семестр»

Студент группы ПИ-54 Дворников М. С.

Преподаватель доцент, к.т.н. Егорова Е.В.

Барнаул 2026

Задания 1-2. (№6, №3). Написать функцию, которая для заданного аргумента x и заданной точности ϵ вычисляет значение $y=f(x)$ как сумму членов бесконечного ряда.

N6.

$$Y(x) = \Gamma_2(x) = \frac{x^2}{2^2 \cdot 2!} - \frac{x^4}{2^4 \cdot 1! \cdot 3!} + \frac{x^6}{2^6 \cdot 2! \cdot 4!} - \frac{x^8}{2^8 \cdot 3! \cdot 5!} \dots$$

N3.

$$Y(x) = \arcsin x = x + \frac{x^3}{2 \cdot 3} + \frac{1 \cdot 3 \cdot x^5}{2 \cdot 4 \cdot 5} + \frac{1 \cdot 3 \cdot 5 \cdot x^7}{2 \cdot 4 \cdot 6 \cdot 7} \dots$$

Текст функций 1, 2 и 3:

```
double row_6(double x, double e) {
    double y = 0; // Сумма ряда
    int n = 0; // Номер члена ряда с нуля
    double up = x * x; // Числитель дроби (начинаем с x^2)
    double down = (2 * 2) * (1 * 2); // Знаменатель дроби (начинаем с 2^2 * 2!)
    int k = 1; // Знак перед членом ряда (1 или -1)
    double cur_member = up / down; // Текущий член ряда (n = 0)
    double prev_member = cur_member * 10; // Предыдущий член ряда (делаем его больше
текущего)
    while ((fabs(cur_member) >= e) && prev_member > fabs(cur_member)) {
        y += cur_member; // Добавляем текущий член ряда
        prev_member = fabs(cur_member); // Сохраняем модуль текущего члена
        n++; k *= -1; // Увеличиваем номер члена и меняем знак
        up *= x * x; // Домножаем числитель на x^2
        down *= (2 * 2) * n * (n + 2); // Домножаем знаменатель на 2^2 и увеличиваем
факториалы
        cur_member = k * up / down; // Получаем текущий член
    }
    return y;
}

double row_3(double x, double e) {
    double y = 0; // Сумма ряда
    int n = 0; // Номер члена ряда с нуля
    double up = x; // Числитель дроби (начинаем с x)
    double down = 1; // Знаменатель дроби (начинаем без знаменателя)
    int k = 1; // Нечётный множитель/делитель члена ряда (1, 3, 5...)
    double cur_member = x; // Текущий член ряда (n = 0)
    double prev_member = cur_member * 10; // Предыдущий член ряда (делаем его больше
текущего)
    while ((fabs(cur_member) >= e) && prev_member > fabs(cur_member)) {
        y += cur_member; // Добавляем текущий член ряда
        prev_member = fabs(cur_member); // Сохраняем модуль текущего члена
        n++; // Увеличиваем номер члена ряда
        up *= x * x * k; // Домножаем числитель на x^2 и k (т.е. делитель n-1)
        k += 2; // Увеличиваем нечётный множитель (теперь это делитель)
        down *= (2 * n); // Домножаем знаменатель на 2 * n (2, 4, 6...)
        cur_member = up / (down * k); // Делим числитель на знаменатель и k
    }
    return y;
}

void row_print(double x, double e, double y, int len) {
    printf("\nДля x = %.1f и e = %.1f:\n", len, x, len, e);
}
```

```
    printf("y = Y(x) = %.*lf\n", len, y);
}
```

Текст программы 1:

```
#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском

#define LEN 6 // Точность вывода чисел

// 1. Функция для вычисления ряда %6 (первый выданный номер)
double row_6(double x, double e);

// 2. Функция для вычисления ряда %3 (второй выданный номер)
double row_3(double x, double e);

// 3. Функция для вывода получившихся значений
void row_print(double x, double e, double y, int len = LEN);

int main() {
    SETIO;
    system("color F0");
    do {
        system("cls");
        // Ввод x
        printf("Введите x из диапазона [0, 1]: ");
        double x;
        do scanf("%lf", &x);
        while ((x < 0) || (1 < x));
        // Ввод e - точность
        printf("Введите точность e из диапазона (0, 1): ");
        double e;
        do scanf("%lf", &e);
        while ((e <= 0) || (1 <= e));
        // Ввод n - номер формулы
        printf("Введите по какой формуле считать y:\n");
        printf("1 - Г2(x)\n");
        printf("2 - arcsin(x)\n");
        int n;
        do scanf("%d", &n);
        while (n != 1 && n != 2);
        // Посчитаем y
        double y;
        switch (n) {
            case 1: { y = row_6(x, e); break; }
            case 2: { y = row_3(x, e); break; }
        }
        row_print(x, e, y); // Выводим результаты
        printf("\nНажмите ESC, чтобы закончить.\n"
            "Нажмите любую другую кнопку, чтобы продолжить");
    } while (_getch() != 27);
}
```

Текст программы 2:

```
// 1. Функция для вычисления ряда %6 (первый выданный номер)
double row_6(double x, double e);

// 2. Функция для вычисления ряда %3 (второй выданный номер)
double row_3(double x, double e);

// 4. Функция для получения y через нужную функцию
double count_y(double x, double e,
    double (*func)(double x, double e));

// 3. Функция для вывода получившихся значений
void row_print(double x, double e, double y, int len = LEN);
```

```

int main() {
    SETIO;
    system("color F0");
    do {
        system("cls");
        // Ввод x
        printf("Введите x из диапазона [0, 1]: ");
        double x;
        do scanf("%lf", &x);
        while ((x < 0) || (1 < x));
        // Ввод e - точность
        printf("Введите точность e из диапазона (0, 1): ");
        double e;
        do scanf("%lf", &e);
        while ((e <= 0) || (1 <= e));
        // Ввод n - номер формулы
        printf("Введите по какой формуле считать y:\n");
        printf("1 - Г2(x)\n");
        printf("2 - arcsin(x)\n");
        int n;
        do scanf("%d", &n);
        while (n != 1 && n != 2);
        // Выбираем нужную функцию
        double (*func)(double x, double e); // Указатель на функцию
        func = row_6; // Компилятор ругается без этой строчки :(
        switch (n) { // Сохраним нужную функцию в указателе
            case 1: { func = row_6; break; }
            case 2: { func = row_3; break; }
        }
        double y = count_y(x, e, func); // Считаем y
        row_print(x, e, y); // Выводим результаты
        printf("\nНажмите ESC, чтобы закончить.\n"
            "Нажмите любую другую кнопку, чтобы продолжить");
    } while (_getch() != 27);
}

double count_y(double x, double e, // Передаём аргумент x и точность e
double (*func)(double x, double e)) { // Передаём функцию
double y = (*func)(x, e); // Возвращаем значение y
return y;
}

```

Тесты программ:

Программы одинаково показывают себя на тестах, поэтому в отчёте прикреплены результаты тестирования только программы 2 из-за большей её сложности

```

Введите x из диапазона [0, 1]: 0.2
Введите точность e из диапазона (0, 1): 0.00001
Введите по какой формуле считать y:
1 - Г2(x)
2 - arcsin(x)
1

Для x = 0.200000 и e = 0.000010:
y = Y(x) = 0.004983

Нажмите ESC, чтобы закончить.
Нажмите любую другую кнопку, чтобы продолжить

```

```

Введите x из диапазона [0, 1]: 0.3
Введите точность e из диапазона (0, 1): 0.001
Введите по какой формуле считать y:
1 - Г2(x)
2 - arcsin(x)
2

Для x = 0.300000 и e = 0.001000:
y = Y(x) = 0.304500

Нажмите ESC, чтобы закончить.
Нажмите любую другую кнопку, чтобы продолжить

```

Задания 3-4 (№8). Написать программу, состоящую из нескольких функций. Связь отдельных функций - только через список параметров.

№ 8.

В двумерной матрице (квадратной) упорядочить каждую строку по возрастанию элементов строки.

Затем из элементов главной диагонали найти наибольший и наименьший элемент.

Текст программы 1:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Стандартный вход/выход
#include <locale.h> // Вывод на русском языке
#include <conio.h> // Задержка экрана

#include <string.h> // Работа со строками
#include <math.h> // Математические функции

#include <stdlib.h> // Создание динамических массивов, использование рандома и преобразования строк
#include <windows.h> // Очистка консоли и установка кодировки

#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
#define N 15 // Максимальный размер матрицы

// 1. Ввод двумерной целочисленной матрицы
void matrix_input(int matrix[N][N], int size = N);

// 2. Вывод двумерной целочисленной матрицы
void matrix_output(int matrix[N][N], int size = N);

// 3. Обработка двумерной целочисленной матрицы по заданию
void martix_sort(int matrix[N][N], int size = N);

// 4. Поиск наибольшего и наименьшего элементов на гл. диаг.
void matrix_maxmin(int matrix[N][N], int size, int* p_max, int* p_min);

int main() {
    SETIO;
    system("color F0");
    do {
        system("cls");
        // Ввод размера матрицы n
        printf("Введите размер квадратной матрицы n (1 <= n <= %d): ", N);
        int n;
        do scanf("%d", &n);
        while ((n < 1) || (N < n));
        int matrix[N][N];
        // Ввод целочисленной матрицы
        printf("Введите матрицу:\n");
        matrix_input(matrix, n);
        // Вывод целочисленной матрицы
        printf("\nПолучилась матрица:\n");
        matrix_output(matrix, n);
        // Сортировка матрицы и её вывод
        martix_sort(matrix, n);
        printf("\nПосле сортировки получилась матрица:\n");
        matrix_output(matrix, n);
        // Поиск наибольшего и наименьшего элементов на гл. диаг.
```

```

        int max, min;
        int* p_max = &max, * p_min = &min;
        matrix_maxmin(matrix, n, p_max, p_min);
        printf("Поиск по главной диагонали:\n");
        printf("Наибольший элемент - %d\n", max);
        printf("Наименьший элемент - %d\n", min);

        printf("\nНажмите ESC, чтобы закончить.\n"
               "Нажмите любую другую кнопку, чтобы продолжить");
    } while (_getch() != 27);
}

void matrix_input(int matrix[N][N], int size) {
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            scanf("%d", &matrix[i][j]);
}

void matrix_output(int matrix[N][N], int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            printf("%d ", matrix[i][j]);
        printf("\n");
    }
}

void martix_sort(int matrix[N][N], int size) {
    for (int i = 0; i < size; i++)
        for (int k = 1; k < size; k++)
            for (int j = 0; j < size - k; j++)
                if (matrix[i][j] > matrix[i][j + 1]) {
                    int buffer = matrix[i][j];
                    matrix[i][j] = matrix[i][j + 1];
                    matrix[i][j + 1] = buffer;
                }
}

void matrix_maxmin(int matrix[N][N], int size, int* p_max, int* p_min) {
    *p_max = matrix[0][0]; // Наибольший элемент гл. диаг.
    *p_min = matrix[0][0]; // Наименьший элемент гл. диаг.
    for (int i = 0; i < size; i++) {
        if (matrix[i][i] > *p_max) *p_max = matrix[i][i];
        if (matrix[i][i] < *p_min) *p_min = matrix[i][i];
    }
}

```

Текст программы 2:

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Стандартный вход/выход
#include <locale.h> // Вывод на русском языке
#include <conio.h> // Задержка экрана

#include <string.h> // Работа со строками
#include <math.h> // Математические функции

#include <stdlib.h> // Создание динамических массивов, использование рандома и преобразования строк
#include <windows.h> // Очистка консоли и установка кодировки

#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
#define N 15 // Максимальный размер матрицы

// 1. Ввод двумерной целочисленной матрицы

```

```

void matrix_input(int* matrix, int size = N);

// 2. Вывод двумерной целочисленной матрицы
void matrix_output(int* matrix, int size = N);

// 3. Обработка двумерной целочисленной матрицы по заданию
void martix_sort(int* matrix, int size = N);

// 4. Поиск наибольшего и наименьшего элементов на гл. диаг.
void matrix_maxmin(int* matrix, int size, int* p_max, int* p_min);

int main() {
    SETIO;
    system("color F0");
    do {
        system("cls");
        // Ввод размера матрицы n
        printf("Введите размер квадратной матрицы n (1 <= n <= %d): ", N);
        int n;
        do scanf("%d", &n);
        while ((n < 1) || (N < n));
        int matrix[N][N];
        // Ввод целочисленной матрицы
        printf("Введите матрицу:\n");
        matrix_input((int*)matrix, n);
        // Вывод целочисленной матрицы
        printf("\nПолучилась матрица:\n");
        matrix_output((int*)matrix, n);
        // Сортировка матрицы и её вывод
        martix_sort((int*)matrix, n);
        printf("\nПосле сортировки получилась матрица:\n");
        matrix_output((int*)matrix, n);
        // Поиск наибольшего и наименьшего элементов на гл. диаг.
        int max, min;
        int* p_max = &max, * p_min = &min;
        matrix_maxmin((int*)matrix, n, p_max, p_min);
        printf("Поиск по главной диагонали:\n");
        printf("Наибольший элемент - %d\n", max);
        printf("Наименьший элемент - %d\n", min);

        printf("\nНажмите ESC, чтобы закончить.\n"
               "Нажмите любую другую кнопку, чтобы продолжить");
    } while (_getch() != 27);
}

void matrix_input(int* matrix, int size) {
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            scanf("%d", (matrix + i * N + j));
}

void matrix_output(int* matrix, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            printf("%d ", *(matrix + i * N + j));
        printf("\n");
    }
}

void martix_sort(int* matrix, int size) {
    for (int i = 0; i < size; i++)
        for (int k = 1; k < size; k++)
            for (int j = 0; j < size - k; j++)

```

```

        if (*(matrix + i * N + j) > *(matrix + i * N + j + 1)) {
            int buffer = *(matrix + i * N + j);
            *(matrix + i * N + j) = *(matrix + i * N + j + 1);
            *(matrix + i * N + j + 1) = buffer;
        }
    }

void matrix_maxmin(int* matrix, int size, int* p_max, int* p_min) {
    *p_max = *matrix; // Наибольший элемент гл. диаг.
    *p_min = *matrix; // Наименьший элемент гл. диаг.
    for (int i = 0; i < size; i++) {
        if (*(matrix + i * N + i) > *p_max) *p_max = *(matrix + i * N + i);
        if (*(matrix + i * N + i) < *p_min) *p_min = *(matrix + i * N + i);
    }
}

```

Тесты программ:

Программы одинаково показывают себя на тестах, поэтому в отчёте прикреплены результаты тестирования только программы 2 из-за большей её сложности

Введите размер квадратной матрицы n (1 <= n <= 15): 3

Введите матрицу:

```

9 8 7
4 6 5
1 2 3

```

Получилась матрица:

```

9 8 7
4 6 5
1 2 3

```

После сортировки получилась матрица:

```

7 8 9
4 5 6
1 2 3

```

Поиск по главной диагонали:

Наибольший элемент - 7

Наименьший элемент - 3

Нажмите ESC, чтобы закончить.

Нажмите любую другую кнопку, чтобы продолжить

Введите размер квадратной матрицы n (1 <= n <= 15): 1

Введите матрицу:

```

1

```

Получилась матрица:

```

1

```

После сортировки получилась матрица:

```

1

```

Поиск по главной диагонали:

Наибольший элемент - 1

Наименьший элемент - 1

Нажмите ESC, чтобы закончить.

Нажмите любую другую кнопку, чтобы продолжить